

Modélisation d'Application et Intégration des IA

Ce document définit le cadre méthodologique du développement applicatif moderne, en mettant en avant l'interaction entre les processus métiers classiques et les nouveaux outils d'intelligence artificielle (Spec-kit, Gemini, AI Studio, etc.).

1. Fondements de la Modélisation d'Application

Le cycle de développement repose sur sept piliers fondamentaux garantissant la viabilité et la qualité du produit final.

1.1 Cadrage et Vision Stratégique

- **Définition de la Vision** : Identification du problème métier et établissement d'objectifs **SMART** (Spécifiques, Mesurables, Atteignables, Réalistes, Temporels).
- **Analyse de Faisabilité** : Évaluation des contraintes techniques, budgétaires et temporelles.

1.2 Recherche Utilisateur (UX Research)

- **Interviews et Ateliers** : Extraction des "Pain Points" via un contact direct avec les utilisateurs.
- **Audit de l'Existant** : Analyse des processus actuels (Excel, papier) et identification des besoins non fonctionnels (sécurité, performance).

1.3 Ingénierie des Exigences

- **PRD (Product Requirement Document)** : Référentiel détaillant le périmètre, les règles de gestion et le glossaire.
- **Backlog Produit** : Décomposition en *User Stories*.
- **Spécifications de Tests** : Critères d'acceptation (souvent en formalisme *Gherkin*).

1.4 Modélisation Conceptuelle (Données)

- **Modèle Logique (MLD)** : Traduction du modèle conceptuel vers une structure compatible avec un SGBD.

1.5 Architecture Technique et Dynamique

- **Stack Technique** : Sélection des langages, frameworks et infrastructure Cloud selon les besoins de scalabilité.

1.6 Design d'Expérience (UI/UX)

- **Maquettes High-Fidelity** : Design visuel final et charte graphique.
- **Prototypage Cliquable** : Simulation de l'interface pour tests en conditions réelles.

1.7 Spécifications de Développement et Implémentation

- **Architecture logicielle** : Définition des patterns (MVC, Microservices, Hexagonale) et du DDD (*Domain Driven Design*).
- **Documentation API** : Contrats d'interface (OpenAPI/Swagger).
- **Plan de Tests** : Stratégie de tests unitaires, d'intégration et de non-régression.

2. Cartographie de l'Assistance par IA

Le tableau ci-dessous détaille comment les outils d'IA (Spec-kit, Gemini, Stitch) interviennent sur chaque sous-étape de modélisation.

Sous-étape	Prise en charge par Spec-kit & Méthode
Interviews & Ateliers	Simulation de personas et d'entretiens via Gemini .
Définition de la Vision	Commande /specify pour transformer l'intention en spec.md.
Rédaction du PRD	Génération automatique du spec.md (PRD moderne) via /specify.
Maquettes (Hi-Fi)	Design UI réalisé avec Google Stitch .
Plan de Tests	Génération de phases de tests avec critères précis via /tasks.
Prototypage	Utilisation du mode "Build" d' AI Studio .
Choix de la Stack	Définition explicite via la commande /plan.
Architecture Logicielle	Définition des patterns et flux de données via /plan.
Spécifications Tests	Génération des tâches de tests et critères d'acceptation via /tasks.
Développement	Codage effectif de l'application via la commande /implement.
Audit & Faisabilité	<i>Prise en charge manuelle ou audit externe.</i>

3. Flux de Données et Chaîne de Valeur

Ce cycle décrit comment l'information est réinjectée d'un outil à l'autre pour maintenir une cohérence globale.

Étape	Flux de données (Réinjection)	Outil & Méthode
Cadrage Initial	Génère le contexte brut (Pain Points), les personas et interviews.	Gemini
Constitution	Utilise le contexte Gemini pour fixer les règles générales.	Spec-kit /constitution
Spécification	Transforme le contexte validé en document spec.md.	Spec-kit /specify
Design Visuel	Injection du spec.md pour créer l'identité visuelle.	Google Stitch
Prototypage	Injection du spec.md et de la maquette pour créer le prototype.	AI Studio
Architecture	Réinjection des retours du prototype dans le plan technique.	Spec-kit /plan
Tâches & MLD	Découpe le projet en tâches basées sur le plan validé.	Spec-kit /tasks
Implémentation	Code l'application en respectant toute la chaîne amont.	Spec-kit /implement